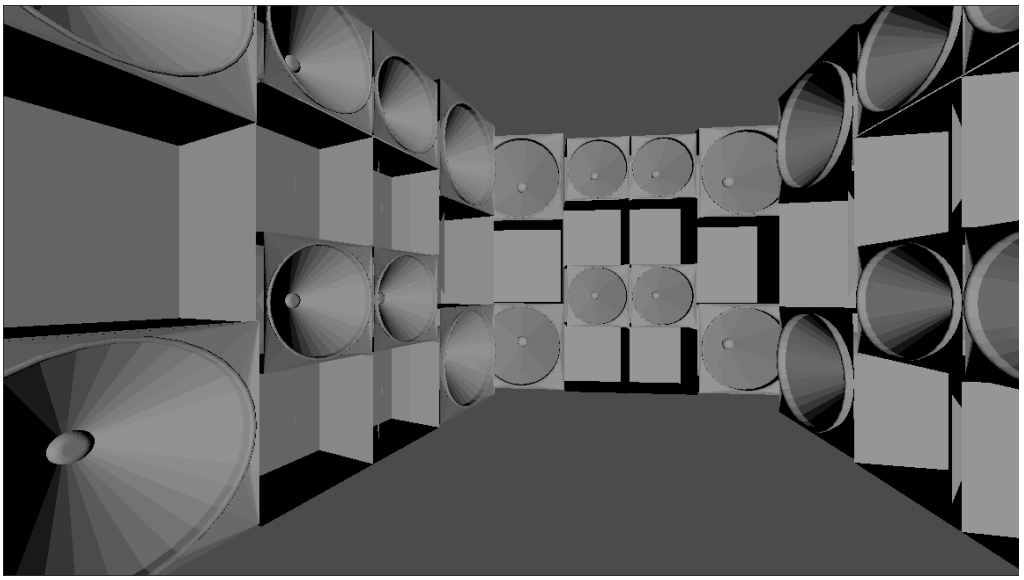
Lucas da Silva Machado
Unisinos 2026 / 02
Animação Computadorizada

Trabalho do Grau A

Para o projeto, o primeiro pensamento foi construir uma espécie de windows media player, porém achei o modelo de auxílio disponibilizado e apresentado em aula muito semelhante ao que eu queria fazer.
Então pensei em algo um pouco diferente, mas que ainda tivesse uma ideia parecida. Como não sou o maior conhecedor da Godot, pedi auxílio de tutoriais do youtube e do Gemini para corrigir alguns bugs e vou documentar abaixo o passo a passo do projeto.

Primeiro passo foi montar uma cena simples, e como o tema música ainda estava na minha cabeça, pensei nos efeitos visuais da Furacão 2000 (sou trinta mais, o que explica a referência). No caso era um paredão de caixas de som montadas no palco, que ao passar da apresentação tinha suas cores sendo alteradas.
Então busquei um OBJ que fosse um paredão de caixas de som, montei uma cena formando uma parede, e parti para a programação.





Criei um código para criar um degradê de cores ao clicar a tecla 1, para dar o resultado visual que eu queria.

```gdscript
# Interação com a tecla 1
func _unhandled_input(event: InputEvent) -> void:
    if event is InputEventKey and event.pressed and not event.echo:
        if event.keycode == KEY_1 or event.keycode == KEY_KP_1:
            mudar_degrade_aleatorio()
```

Aqui é a implementação da mudança da função de mudança de degradê.
Um problema que tive no começo foi que a textura estava alterando apenas um triângulo da malha, então o modelo não trocava de cor. Nesse caso pedi auxilio do Gemini que me passou o passo a passo da criação e implementação da textura gradiente.

```gdscript
# Função para aleatorizar as cores
func mudar_degrade_aleatorio() -> void:

    var cor_inicial = Color(randf(), randf(), randf())
    var cor_final = Color(randf(), randf(), randf())

    # Criar recurso de Gradiente na Godot
    var gradiente = Gradient.new()
    gradiente.set_color(0, cor_inicial)
    gradiente.set_color(1, cor_final)

    # Transformar o gradiente numa textura 2D - Gemini me ajudou nesse função, pois a primeira que eu fiz trocava a cor de apenas 1 triangulo.
    var textura_degrade = GradientTexture2D.new()
    textura_degrade.gradient = gradiente

    textura_degrade.fill_from = Vector2(randf(), randf())
    textura_degrade.fill_to = Vector2(randf(), randf())

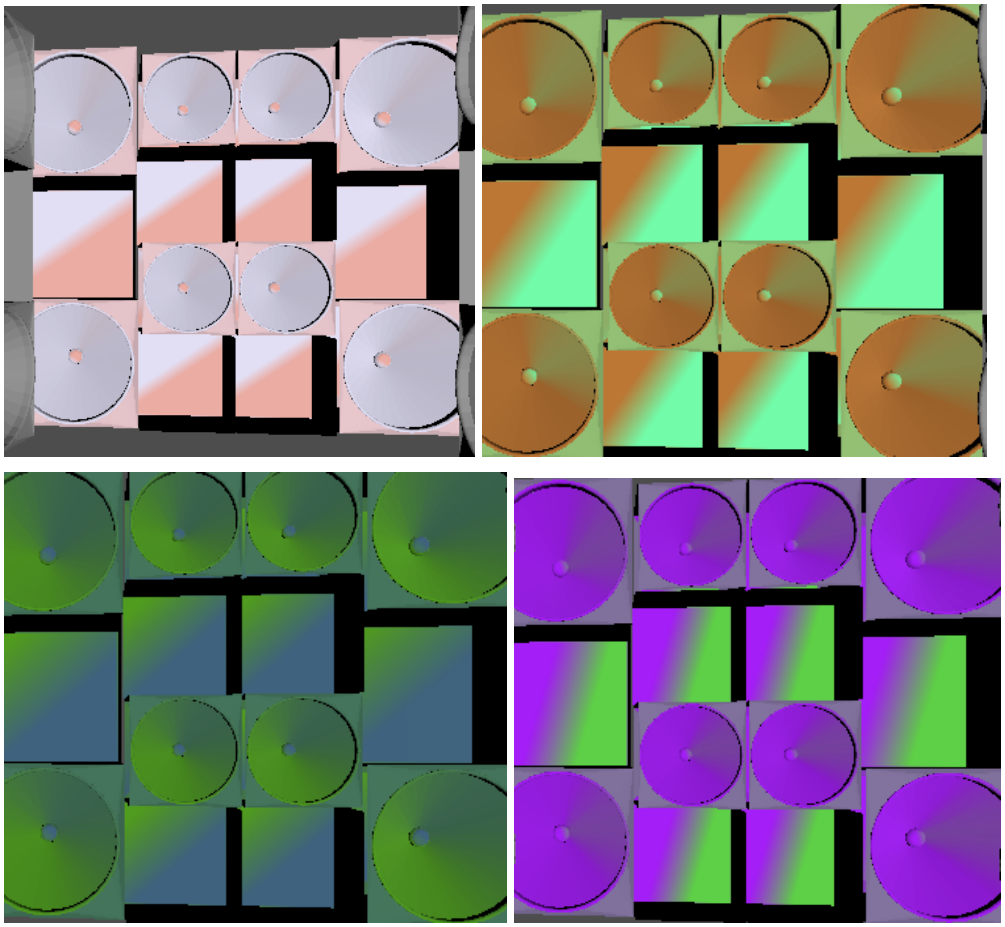
    # Aplicar textura no objeto
    aplicar_textura_recursivo(self, textura_degrade)
```

Depois o código precisa percorrer pelo obj aplicando o novo material, recebendo como parâmetro o nó e a textura degradê.

```gdscript
#funcao para as validacoes e aplicacoes das texturas, recebe a textura e o nó da cena para fazer as mudanças
func aplicar_textura_recursivo(no: Node, textura: Texture2D) -> void:
    if no is MeshInstance3D and no.mesh: #validação se é um objeto 3D, afim de evitar erros
        for surface_idx in range(no.mesh.get_surface_count()): #percorrer a superfície do obj
            var mat = no.get_surface_override_material(surface_idx)
            #validação se possui material, não vai funciona se não criar um material do zero nos modelos que importei
            if mat == null:
                var mat_ativo = no.get_active_material(surface_idx)
                if mat_ativo:
                    mat = mat_ativo.duplicate()
                else:
                    mat = StandardMaterial3D.new()
                no.set_surface_override_material(surface_idx, mat)

            if mat is StandardMaterial3D or mat is ORMMaterial3D:
                # Reseta a cor para branco puro, sem esse passo o degradê fica estranho
                mat.albedo_color = Color.WHITE
                # Aplica a textura do degradê
                mat.albedo_texture = textura
```

O resultado obtido se encontra selecionando a tecla 1 do teclado:



Perfeito, esse é o código base que foi deixado em uma caixa de som pequeninha no cenário para que fique guardado, agora o próximo passo é mudar a aleatoriedade de acordo com a batida da música. Ou seja, quero que as cores se alterem de acordo com os dados da faixa.

Agora a ideia é fazer com a batida da música o objeto mudar de cor, emitir partícula na cor random e pulsar de acordo com o som. Primeiro é iniciada todas as configurações/variáveis necessárias ao longo do código. Isso fora das funções.

```gdscript
extends Node3D

# Refs e Variáveis
var music_player: AudioStreamPlayer
var spectrum_analyzer: AudioEffectSpectrumAnalyzerInstance

## Nó de partículas para emitir.
@export var emissor_particulas: Node3D

## Raio de emissão das particulas
@export var raio_campo_emissao: float = 2.2

# Configurações para o efeito tremido da caixa de som
@export var multiplicador_escala_batida: float = 1.12
@export var velocidade_retorno_escala: float = 3.5
var escala_original: Vector3 = Vector3.ONE

# Constantes de áudio
const FREQ_MAX: float = 11050.0
const MIN_DB: float = 60.0
const FFT_SIZE: int = 2048
var hz_por_bin: float = 0.0
```

```gdscript
# Detectores da batida (BASS / GRAVES)
@export var freq_min_batida: float = 20.0
@export var freq_max_batida: float = 150.0 #número seguro para que o efeito não vire um strobo
@export var limiar_batida: float = 0.35
@export var tempo_recarga_batida: float = 0.15
var tempo_desde_ultima_batida: float = 0.0

# Variáveis para as cores do efeito de degradê
var cor_inicial: Color = Color.BLACK
var cor_final: Color = Color.WHITE
```

Depois criei uma função para garantia do obj sendo iniciado corretamente.
No ready é garantido que toda iniciação ocorrerá bem. Aqui é salvo a escala do obj vinculado, gerado uma instancia de áudio, chamado as funções de inicialização das partículas e atualização do degrade, carregamento da faixa de audio:

```gdscript
# Função de prepação para garantir que o objeto seja carregado corretamente
func _ready() -> void:
    escala_original = scale #salva o tamanho original da escala

    if has_node("MusicPlayer"): #garantir que o nó de audio exista dentro do código para realizar as comunicações
        music_player = $MusicPlayer as AudioStreamPlayer
    else:
        music_player = AudioStreamPlayer.new() #nova instancia de audio
        music_player.name = "MusicPlayer"
        add_child(music_player)

    configurar_emissor_particulas() #função para preparar as particular, explicada abaixo

    spectrum_analyzer = AudioServer.get_bus_effect_instance(0, 0) #captar a frequencia que está no bus Master da Godot, para ler os graves em tempo real.

    if not spectrum_analyzer:
        push_error("ERRO: Efeito SpectrumAnalyzer não encontrado no bus Master (índice 0, efeito 0).") #código de erro por segurança
        return

    var nyquist: float = AudioServer.get_mix_rate() / 2.0 #calculo da frequencia que usa a taxa (Nyquist) e o tamanho da FFT(2048) para descobrir os HERTZ
    hz_por_bin = nyquist / (FFT_SIZE / 2.0)
```

```gdscript
if music_player.stream == null: #Carrega a faixa de audio
    if ResourceLoader.exists("res://batida.ogg"): #n'ao esquecer de colocar o nome do arquivo certo, se n'ao da erro
        music_player.stream = load("res://batida.ogg")
    else:
        push_warning("Aviso: O arquivo 'res://batida.ogg' não foi encontrado na raiz do projeto.")

cor_inicial = Color(randf(), randf(), randf()) #sorteia a cor 1 do degrade
cor_final = Color(randf(), randf(), randf()) #sorteia a cor 2 do degrade
atualizar_degrade() #atualiza a funcao degrade

if music_player.stream: #reproduz o audio, tbm n'ao da para esquecer se n'ao sem musica
    music_player.play()
```

Já a função process serve para controle dos fluxos e intensidade das batidas, cronometrando os tempos para que uma pulsão não sobreponha a outra, e se a música está gerando intensidade para que o pulso aconteça.

```gdscript
#funcao para realizar o processo de "inflar e desinflar" o obj, de forma suave
func _process(delta: float) -> void:
    tempo_desde_ultima_batida += delta #cronometro mede o tempo decorrido a cada frame de pulso
    scale = scale.lerp(escala_original, delta * velocidade_retorno_escala) #código para transicionar levemente as escalas dos objs

    if not spectrum_analyzer: #segurança caso der erro na leitura
        return

    #mede com o analisador de audio da Godot a quantidade de som emitida
    var magnitude_vetor: Vector2 = spectrum_analyzer.get_magnitude_for_frequency_range(freq_min_batida, freq_max_batida)
    #converte para 0 e 1.0 a intensidade bruta captada pelo canal de audio no instante
    var energia_batida: float = calcular_energia_normalizada(magnitude_vetor.length())

    #valida se a intensidade bateu o limite exigido e valida se ta dentro do tempo do cronometro para impulsionar uma nova batida
    if energia_batida >= limiar_batida and tempo_desde_ultima_batida >= tempo_recarga_batida:
        tempo_desde_ultima_batida = 0.0
        processar_nova_batida()
```

Em processar_nova_batida é passado os dados das novas cores e escalas, a chamada as funções e atualização do degrade e emissão de particular.

```gdscript
#função para processamento da nova batida, passando a cores e escalas da atualização
func processar_nova_batida() -> void:
    cor_inicial = cor_final
    cor_final = Color(randf(), randf(), randf())

    scale = escala_original * multiplicador_escala_batida

    atualizar_degrade()
    emitir_particulas()
```

Logo em seguida temos a função de emissão das partículas. Aqui é feito teste se o nó está na CPU ou GPU e também é captada a cor random do process.material para que a partícula fique com a mesma cor do pulso.

```
# Função para disparar as partículas brilhantes
func emitir_particulas() -> void:
    if emissor_particulas == null: #validador de segurança
        return

    #se for um nó de particulas na GPU
    if emissor_particulas is GPUParticles3D:
        var gpu_p = emissor_particulas as GPUParticles3D #converte para o tipo correto de GPU e libera o acesso as suas propriedades

        #altera a cor para combinar com a cor das caixas de som
        if gpu_p.process_material is ParticleProcessMaterial:
            (gpu_p.process_material as ParticleProcessMaterial).color = cor_final
        #altera a cor do material 3D desenhado na tela
        if gpu_p.draw_pass_1 and gpu_p.draw_pass_1.material is StandardMaterial3D:
            (gpu_p.draw_pass_1.material as StandardMaterial3D).albedo_color = cor_final
        gpu_p.restart() #reinicia o ciclo e força do disparo

    #se for um nó de particulas na CPU, algumas mudanças mas a estrutura é a mesa
    elif emissor_particulas is CPUParticles3D:
        var cpu_p = emissor_particulas as CPUParticles3D
        cpu_p.color = cor_final

        if cpu_p.mesh and cpu_p.mesh.material is StandardMaterial3D:
            (cpu_p.mesh.material as StandardMaterial3D).albedo_color = cor_final

        cpu_p.restart()
```

Já na função configurar emissor de partículas, o código válida se está vazio, e cria o nó. Também é nessa função que é instanciado os efeitos visuais das partículas.

```
#Funcao chamada no momento que a batida é detectada, coordena as questões visuais do OBJ
func configurar_emissor_particulas() -> void:
    if emissor_particulas != null:
        return

    for filho in get_children(): #percorrer todos nós filhos
        if filho is GPUParticles3D or filho is CPUParticles3D:
            emissor_particulas = filho #define o nó como emissor ativo
            return

    var novas_particulas = CPUParticles3D.new()
    novas_particulas.name = "ParticulasBatida"
    novas_particulas.position = Vector3(0, 3, 0)
    novas_particulas.one_shot = true
    novas_particulas.explosiveness = 0.95
    novas_particulas.amount = 50
    novas_particulas.lifetime = 0.4

    # Campo de emissão em formato de esfera
    novas_particulas.emission_shape = CPUParticles3D.EMISSION_SHAPE_SPHERE
    novas_particulas.emission_sphere_radius = raio_campo_emissao
```

```
novas_particulas.spread = 180.0
novas_particulas.initial_velocity_min = 3.0
novas_particulas.initial_velocity_max = 8.0
novas_particulas.gravity = Vector3(0, -2, 0) # aplicar gravidade simulada tombarem naturalmente
novas_particulas.scale_amount_min = 0.5
novas_particulas.scale_amount_max = 1.8
novas_particulas.emitting = false

#Usa QuadMesh em vez de esfera (plano 2D)
var mesh_quad = QuadMesh.new()
mesh_quad.size = Vector2(0.2, 0.2)

#Material Aditivo e Billboard, sempre olhando pra câmera
var mat_particula = StandardMaterial3D.new()
mat_particula.transparency = BaseMaterial3D.TRANSPARENCY_ALPHA
mat_particula.blend_mode = BaseMaterial3D.BLEND_MODE_ADD # Mistura aditiva para dar efeito de Luz/Glow
mat_particula.shading_mode = BaseMaterial3D.SHADING_MODE_UNSHADED
mat_particula.billboard_mode = BaseMaterial3D.BILLBOARD_PARTICLES # Vira para a câmera automaticamente
mesh_quad.material = mat_particula

novas_particulas.mesh = mesh_quad

add_child(novas_particulas)
emissor_particulas = novas_particulas
```

É preciso atualizar o degrade da textura nas cores do OBJ, isso é feito na função actualizar_degrade().

```
#Montagem  da textura do degradê com as novas cores no OBJ
func atualizar_degrade() -> void:
    var gradiente = Gradient.new()
    gradiente.set_color(0, cor_inicial)
    gradiente.set_color(1, cor_final)

    var textura_degrade = GradientTexture2D.new()
    textura_degrade.gradient = gradiente
    textura_degrade.fill_from = Vector2(randf(), randf())
    textura_degrade.fill_to = Vector2(randf(), randf())

    aplicar_textura_recursivo(self, textura_degrade)
```

A função de textura recursiva faz também uma validação da mash vazia, após isso busca se o material está ativo e o duplica para realizar as mudanças pertinentes, aplicando a textura nova no nó.

```
# Aplica material e textura de forma recursiva na cena
func aplicar_textura_recursivo(no: Node, textura: Texture2D) -> void:
    if no is MeshInstance3D and no.mesh:
        for surface_idx in range(no.mesh.get_surface_count()):
            var mat = no.get_surface_override_material(surface_idx)

            if mat == null:
                var mat_ativo = no.get_active_material(surface_idx) #buscar o material ativo do OBJ
                if mat_ativo:
                    mat = mat_ativo.duplicate() #duplicação é boa pratica caso precise reutilizar o material em outro OBJ
                else:
                    mat = StandardMaterial3D.new() #se não existir material, se é criado um novo.
                no.set_surface_override_material(surface_idx, mat)

            if mat is StandardMaterial3D or mat is ORMMaterial3D:
                mat.albedo_color = Color.WHITE #reseta cor base
                mat.albedo_texture = textura #coloca textura do degrade

    for filho in no.get_children(): #percorre pelos nós caso tenha
        if filho == emissor_particulas:
            continue
        aplicar_textura_recursivo(filho, textura)
```

Conversor de decibeis:

```
# Converte a magnitude linear em Decibéis (dB) e normaliza [0.0, 1.0]
func calcular_energia_normalizada(magnitude_linear: float) -> float:
    if magnitude_linear <= 0.0:
        return 0.0

    var magnitude_db: float = linear_to_db(magnitude_linear)
    return clampf((MIN_DB + magnitude_db) / MIN_DB, 0.0, 1.0)
```

Percorre um intervalo de frequência, e faz a conversão dos hertz, retornando as energias

```
# BLOCO DAS FUNÇÕES AUXILIARES PARA RODAR O CÓDIGO
# Função que recebe as frequencias mínimas e máximass do Hertz
func getAmplitudeInFrequencyRange(min_hz: float, max_hz: float) -> Array[float]: ...

func getSpectrumFrequenciesInRange(min_hz: float, max_hz: float, channel: int = -1) -> Array[float]: ...

func getSpectrumFrequencyBands(n_bands: int) -> Array[float]: ...

func getBandFrequencies(n_bands: int) -> Array[float]: ...
```

Resultado final:

